UniTe: A Universal Tensor Abstraction for Capturing Spatial Relationships

JESSICA M. RAY*, MIT Lincoln Laboratory, USA
TEODORO COLLIN, MIT CSAIL, USA
VIVIENNE SZE, MIT RLE, USA
ALBERT REUTHER, MIT Lincoln Laboratory, USA
SAMAN AMARASINGHE, MIT CSAIL, USA

Tensors are an integral part of numerous domains, and while significant effort has been put into the design of tensor data structures in isolation, little attention has been paid to the relationships that exist across tensors and how this affects their representation and use. In this paper, we focus on spatial relationships across tensors in a program, where such tensors are defined relative to a common reference coordinate system. These relationships are complicated by the fact that the tensors may differ in their representations, such as having variations in their axes, spacings, origins, and overall shape. Due to the lack of existing abstractions and language support for these types of tensor semantics, users are currently forced to manually perform the bookkeeping necessary to account for these varying relationships and representations. Unfortunately, we cannot rely on a simple library to capture these relationships, as computations on these types of tensors often happen at the innermost levels of programs; we find that the overheads associated with an unoptimized implementation quickly accumulate, leading to performance up to nearly 65x slower than a reference C implementation on a series of image and video compression benchmarks.

In this paper, we introduce the novel UniTe abstraction, which captures spatial relationships across all such tensors in a program. We also introduce two domain-specific languages and optimizing compilers, CoLa for Python and SHiM for C/C++, built off of UniTe. Both CoLa and SHiM provide users an intuitive set of tensor primitives based on spatial relationships, hiding the complexity that goes into maintaining the tensors and computing accesses across them. In addition, we discuss the optimizations necessary to remove the associated abstraction overhead, and describe their implementations. On the benchmarks, we show that both CoLa and SHiM successfully remove the overheads, achieving performance parity with existing C implementations.

CCS Concepts: • **Software and its engineering → Domain specific languages**; **Compilers**.

ACM Reference Format:
Jessica M. Ray, Teodoro Collin, Vivienne Sze, Albert Reuther, and Saman Amarasinghe. 2025. UniTe: A Universal Tensor Abstraction for Capturing Spatial Relationships. 1, 1 (December 2025), 25 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Arrays were the only high-level data structure introduced with the first high-level programming language, FORTRAN [Backus 1978]. Since FORTRAN, the N-dimensional array, or tensor, has become the indispensable data structure at the heart of many domains from scientific computing to machine learning, as well as many popular programming languages, such as APL, MATLAB, Halide, Julia, PyTorch, and TensorFlow [Abadi et al. 2015; Bezanson et al. 2017; Iverson 1991; Ragan-Kelley et al. 2013a; Wu 2023]. Moreover, many programmers count on dense N-dimensional arrays to provide a combination of ease and performance. Across domains, programmers know many tricks to confidently

*Work completed while at MIT CSAIL.
New Paper, Not an Extension of a Conference Paper

Authors' Contact Information: Jessica M. Ray, jray@csail.mit.edu, MIT Lincoln Laboratory, Lexington, MA, USA; Teodoro Collin, teoc@csail.mit.edu, MIT CSAIL, Cambridge, MA, USA; Vivienne Sze, sze@mit.edu, MIT RLE, Cambridge, MA, USA; Albert Reuther, reuther@ll.mit.edu, MIT Lincoln Laboratory, Lexington, MA, USA; Saman Amarasinghe, samana@csail.mit.edu, MIT CSAIL, Cambridge, MA, USA.

2025. Manuscript submitted to ACM

express applications with tensors; when programmers use these tricks, they often count on decent baseline performance as many compiler analyses (such as Array Data Dependence Analysis and Polyhedral Analysis [Maydan et al. 1993; Pugh 1992; Verdoolaege 2010]) were created to get performance from array-based codes written in the idiom of array programming. However, the tensor data structure abstraction – *a single uniform rectilinear grid of values with integer coordinates* – has not changed or evolved much over the last half a century.

Most programs normally contain more than one tensor data structure, and in many cases, these tensors are implicitly related to a single reference coordinate system. Programmers draw on the rich history of coordinate systems (such as in physical simulation [Margenau and Murphy 2009]) where tensors representing related data differ by axes, spacings, and/or origins. These tensors have a *spatial relationship* to the reference coordinate system and, thus, to each other.

Despite the ubiquity of spatial relationships, currently, there is no programming language or compiler support for them. Although languages such as Halide [Ragan-Kelley et al. 2013a] and libraries such as Numpy[1] [Harris et al. 2020] have sophisticated support for arrays, users must still manually write out formulas to compute index relationships between multiple arrays for anything more complex than two arrays related by a reshape. In particular, if the indices of two arrays are related spatially via some indexing function and then we reshape one of the arrays, we know of no language or compiler that saves us from having to rewrite the indexing function. Programmers have long struggled with this manual approach to spatial mappings. For example, unit conversions are a well-known source of bugs [Oberg 1999; Taylor et al. 2021], epitomized by one of the most famous bugs in programming languages history: the Mars Climate Orbiter crash [Oberg 1999]. In a spatial mapping, unit conversions are simply different spacings of the coordinate space grid mapped to each tensor. In domains such as graphics and unmanned vehicles, coordinate systems are considered a more general class of bugs [Geisler et al. 2020; Taylor et al. 2021].

To ensure that programmers use spatial relationships correctly, we could encode spatial relationships naively into data structures. However, naively encoding spatial relationships requires computing many coordinate and inverse coordinate transformations, which are often within the innermost loop of a program, leading to large overheads. Thus, properly incorporating spatial mappings requires not only programming language extensions, but compiler support as well.

In this paper, we introduce the UniTe (Universal Tensor) abstraction, the first comprehensive framework for capturing spatial relationships between tensors in a manner that exposes the structure of the spatial relationships to the compiler. We claim the following contributions:

- We define UniTe, a mathematical model of spatial relationships.
- We show how to build these spatial relationships into a tensor abstraction.
- We show how compilers can support and reason about this new abstraction.
- We implement two DSLs and optimizing compilers, CoLa (Python) and SHiM (C++), that leverage the UniTe abstraction for spatial operations. In particular, we implement and discuss the types of optimizations necessary to reduce overheads that may occur.
- We use compression as a case study to demonstrate the impact of the UniTe abstraction for simplicity without compromising performance. In particular, we show that in a video compression application, a simple naive implementation of spatial relationships incurs up to 65× slowdown when compared to reference C implementations, whereas an implementation in our DSLs achieves parity.

[1] Including tools that jit numpy such as Codon or Numba[Lam et al. 2015; Shajii et al. 2023]

- We discuss further application of the UniTe abstraction to data movement within memory hierarchies and distributed compute.

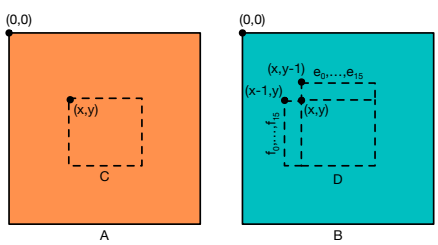
This paper begins with a case study in Section 2 highlighting spatial relationships that exist within one stage of video compression. Section 3 introduces UniTe, and Section 4 defines several operations built on top of UniTe primitives. Section 5 discusses domain-specific compiler optimizations for UniTe, while Sections 6 and 7 describe the CoLa and SHiM DSLs, respectively. Section 8 discusses the application of the UniTe abstraction to data movement operations in memory hierarchies and distributed memory. Finally, Section 9 discusses existing array-centric work.

2 Case Study: Spatial Relationships in Video Compression

This section walks through one practical usage of spatial relationships in the context of video compression. In particular, we look at an operation called intra-prediction, which forms an integral component of many common types of video compression, including H.264 [ITU-T 2021a], H.265 [ITU-T 2021b], and H.266 [ITU-T 2022]. Intra-prediction heavily relies on accessing pixels *at the same location* across multiple tensors, and Figure 1a gives a high-level view of these accesses[2]. The six ordered steps shown below the tensors each represent a different access utilizing some form of spatial relationship across the tensors (we utilize a NumPy-like syntax to represent each operation[3]):

(1) Create two $X \times Y$ tensors, A and B, with the same location in the reference coordinate space.
(2) Create a 16×16 view representing the subtensor, C, relative to A with origin (x, y).
(3) Create a 16×16 view representing the subtensor, D, relative to B that corresponds to the location of C.
(4) Access the elements e_i at indices $(i, -1)$ relative to D. This corresponds to the row above D, where each element has the origin $(x + i, y - 1)$ relative to B.
(5) Access the elements f_i at indices $(-1, i)$ relative to D. This corresponds to the column to the left of D, where each element has the origin $(x - 1, y + i)$ relative to B.

[2]Note that the origins of the tensors are given by the coordinate at the upper-left corner.
[3]Section 4 gives a more concrete description of syntax.




```
(1) A,B = size X by Y tensors
(2) C = A[x:x+16,y:y+16]
(3) D = B[C]
(4) e_i = D(i,-1)
(5) f_i = D(-1,i)
(6) C[i,j] = combine(e_i, f_i)
```

(a)

```
void intra_pred(T_A &A, T_B &B,
                int x, int y) {
  // Extract views (subtensors)
  auto C = A.extract({x,x+16},{y,y+16});
  auto D = B[C];
  dint H = 0, V = 0;
  for (dint i = 0; i < 8; i=i+1) {
    // e_i elements
    H += (i+1)*(D(8+i,-1) - D(6-i,-1));
    // f_i elements
    V += (i+1)*(D(-1,8+i) - D(-1,6-i));
  }
  dint a = 16 * (D(-1,15) + D(15,-1));
  dint b = (5 * H + 32) >> 6;
  dint c = (5 * V + 32) >> 6;
  C[i][j] = (a+b*(i-7)+c*(j-7)+16) >> 5;
}
```

(b)

Fig. 1. (a) Spatial relationships and accesses across tensors in intra-prediction. The corresponding syntax captures the relationships and shows access order. (b) Using an abstraction for spatial relationships to implement the access logic of a. This code uses the SHiM DSL introduced in Section 7.

(6) Combine the e_is and f_is (based on the calculation in the standard) and write the result back to the region in A represented by C.

With this simple syntax and associated description, it is very easy to represent the accesses relative to different tensors, and with an appropriate abstraction for spatial relationships, we can translate this to a DSL and intuitively implement all the accesses in the entire operation. Figure 1b shows such an implementation, which uses the SHiM C++ DSL (see Section 7). SHiM hides all the bookkeeping and indexing necessary to track the spatial relationships across the various tensors, which makes it not only intuitive to use, but also makes it easy to look at the code and understand how the accesses relate to one another. For example, Lines 9 and 11 make it obvious that we are reading elements in the row above and column to the left of tensor D, respectively. Line 5 shows that we want to access tensor B at C's location. The user also can effectively ignore other implementation details hidden by the abstraction, such as whether A and B are standalone tensors, or if they themselves are views (as this effects how you compute the final index into the actual C++ array backing the tensor).

Now, compare this implementation with that shown in Figure 2, which is from the reference implementation of H.264 video compression [Joint Video Team 2022]. The reference implementation performs the same operation as Figure 1b, but requires significantly more complex code (about 100 lines of additional code are omitted for space). There are several reasons for the complexity, all stemming from the lack of an appropriate abstraction to represent tensors with spatial relationships.

```
// Create 1D tensor holding data required
// for intra-prediction
void set_intrapred_16x16(Macroblock *currMB,
                         ColorPlane pl,
                         int *left_available,
                         int *up_available,
                         int *all_available) {
    ...
    int pos_y = pix_a.pos_y;
    int pos_x = pix_a.pos_x;
    for (i = 1; i < MB_BLOCK_SIZE + 1; ++i)
      PredPel[i + 16] = img_enc[pos_y++][pos_x];
    ...
void getNonAffNeighbour(Macroblock *currMB,
                        int xN, int yN,
                        int mb_size[2],
                        PixelPos *pix) {
  BlockPos *PicPos = currMB->p_Vid->PicPos;
  if (xN < 0) {
    if (yN < 0) {
      pix->mb_addr   = currMB->mbAddrD;
      pix->available = currMB->mbAvailD;
    } else if ((yN >= 0)&&(yN < mb_size[1])) {
      pix->mb_addr   = currMB->mbAddrA;
      pix->available = currMB->mbAvailA;
    }
    ...
void get_i16x16_plane(imgpel **cur_pred,
                      imgpel *PredPel) {
  int i, j;
  int ih=0, iv=0;
  int ib, ic, iaa;
  imgpel *t_pred = &PredPel[25];
  imgpel *u_pred = &PredPel[8];
  imgpel *b_pred = &PredPel[23];
  for (i = 1; i < 8;++i) {
    // e_i elements
    ih += i*(*(u_pred + i) - *(u_pred - i));
    // f_i elements
    iv += i*(*t_pred++ - *b_pred--);
  }
  // additional e_i element
  ih += 8*(*(u_pred + 8) - PredPel[0]);
  // additional f_i element
  iv += 8*(*t_pred++ - PredPel[0]);
  iaa=16 * (PredPel[16] + PredPel[32]);
  ib = (5 * ih + 32) >> 6;
  ic = (5 * iv + 32) >> 6;
   for (j=0;j< MB_BLOCK_SIZE;++j) {
     for (i=0;i< MB_BLOCK_SIZE;++i) {
       cur_pred[j][i] =
           (iaa+(i-7)*ib+(j-7)*ic+16)>>5;
     }
   }
}
```

Fig. 2. Snippet of code from [Joint Video Team 2022] used to implement the logic of Figure 1a. This code must manually track the locations of and compute all underlying indices into each tensor.

First, the entire code base[4] is inconsistent in how it represents tensors. In some instances, the tensors are 1D; in others, they are 2D. In this case, the tensor `PredPel` (which holds the data corresponding to the e_i and f_i elements of Figure 1a) is 1D, but also represents the elements in a disjoint manner. Because of this, the simple mappings of e_i and f_i to D(i,-1) and D(-1,i), respectively, do not apply; instead we are left with the accesses on Lines 33 through 46. Unlike the code in Figure 1b, it is not clear what the location of the accessed data is. For example, the `u_pred` pointer on Line 34 points to the data representing the e_i elements, but determining that requires reading a comment in the code that describes the layout of `PredPel`.

Another source of complexity lies in the need to manually track the relative locations of the required tensors. In this case, this complexity rears its head when computing the locations of the e_i and f_i elements for the purpose of

[4]About 75,000 lines

filling `PredPel`. The snippets of functions `set_intrapred_16x16` and `getNonAffNeighbour` starting on Lines 3 and 14, respectively, provide some examples of this. These functions have to get both the individual pixel locations (e.g. Lines 9, 10, and 12) relative to their containing tensor, as well as the location of some intermediate subtensors (e.g. Lines 21 and 24) relative to their containing tensors (which may also be subtensors themselves, requiring another level of relative computation, and so on). In fact, for one commonly accessed tensor, this code base maintains over 130 different parameters that map the location of that tensor and its pixels relative to other tensors in the program.

Compared to some of the other spatial relationships used throughout video compression, these relationships in this example are relatively "simple" as we assume that the two tensors A and B in Figure 1a have the same shape and origins, meaning all our relative accesses across the tensors can effectively be computed by translating the origin of each tensor (for example, accessing element e_i with D(i,-1) translates D's origin i elements to the right and 1 elements up). However, this is not always the case. In some operations, A and B may have different densities (i.e. spacing between coordinates). For example, A may have 1/16 the density of B, meaning every 16×16 region of pixels in B maps to a single pixel in A, and vice versa (even though logically, they still represent the same location). Other times, tensors could have a different dimensionality, such as A being 3D and B being 2D. Regardless of the difference in representation, these tensors must still maintain a spatial relationship with one another. With an abstraction, these parameterizations can be hidden behind the tensor interface and remain transparent to the user. Without an abstraction, this means even more bookkeeping and manual mappings, increasing the complexity of the code and opening the door for bugs.

3 The UniTe Abstraction

The last section highlighted the utility of using spatial relationships to manipulate data. While it is simple, even intuitive, to describe spatial relationships at a high level, the last section also highlighted the complexity of storing and using these relationships in concrete applications. In this section, we introduce the mathematical foundations of such relationships with the UniTe (Universal Tensor) abstraction, which naturally captures spatial relationships between tensors.

At the core of these foundations lies the idea that many different *relative* representations of the same spatial location can be reduced to a single *absolute* representation, *the reference space*. Unlike other conceptions of coordinates in languages for graphics or finite element methods [Chiw et al. 2012; Geisler et al. 2020; Logg et al. 2012] (where coordinates are parts of algorithms to compute integrals on meshes in a uniform manner), this conception of reference space will focus on reconciling space with tensor computations.

3.1 UniTe Foundations

3.1.1 Reference Spaces. Tensor computations traditionally assign some rectangular subset $\mathcal{R}$ of $\mathbb{Z}^N$ to values in the tensor via an indexing operation. UniTe extends tensors by allowing another rectangular subset $\mathcal{S} \subset \mathbb{Z}^n$ to index rectangular subsets of the *same* tensor. Naturally, we can recover the former view for any given $\mathcal{S}$ via a partial mapping: $T : \mathcal{R} \to \mathcal{P}(\mathbb{Z}^n)$. The map T essentially assigns regions of the original index set of a tensor to tensor locations marked by a subset of the indices $\mathcal{S}$. The inverse partial map, $T^{-1} : \mathcal{P}(\mathbb{Z}^N) \to \mathcal{R}$, recovers the idea of indexing the tensor, supplying indices to get values, wherever the map is defined. Linking the tensor perspective to the UniTe perspective, we want $\mathcal{R}$ to be the physical or absolute locations, while T gives a set of logical or relative locations[5]. Thus, $\mathcal{R}$ is a *reference space* and T represents a way transforming to some other space, $\mathcal{S}$.

[5]We note the impedance mismatch between the typical contrast of physical and logical in databases vs. our desired meaning of absolute and relative; thus, we mean to stick to relative vs. absolute locations to avoid confusion.

The full UniTe abstraction emerges after we motivate two key concerns: we need to have many possible choices of T, but we also need to be able to efficiently compare locations viewed from the different choices of T. First, we definitely require many choices of T, as even in the same application, data may be viewed from several different coordinate systems. Moreover, most applications have a canonical choice for T: different choices are useful even within the same applications. Second, we do not wish to allow for arbitrary T because if we even have two different T_0 and T_1, determining the relationship between an object in $T_0(\mathcal{R})$ and an object in $T_1(\mathcal{R})$ requires an inverse in general as we are, by definition, computing $T_0 \circ T_1^{-1}$ [6]. Practically, to determine if a collection of points $P \in T_0(\mathcal{R})$ is really the same as some collection of $P' \in T_1(\mathcal{R})$ requires either T_0^{-1} or T_1^{-1} as one must be mapped back to the reference space. For arbitrary T (non-affine), this would have to be a runtime computation as inverting functions often requires solving a non-linear optimization problem, which cannot be done symbolically in a reliable fashion. For restricted T, we can compute or simplify T^{-1} symbolically as part of code generation.

Since we must have many choices of T but also retain access to reasonable efficient inverses, we limit our families of T to those defined via *reference spaces*, which are views on a single grid in $\mathbb{Z}^N$, transformed in a limited set of ways, motivated by tensor, compression, image processing, and scientific computing applications: *permutations* reorder the axes of a grid, *translations* shift the origin of the grid, *slices* drop axes from the grid, *refinements* uniformly increase the density (spacing) of points in the grid, and *coarsenings* uniformly decrease the density (spacing) of points in the grid.

Moreover, in our definitions of reference spaces (Theorem 3.1), we define a trie where each node is a grid in $\mathbb{Z}^N$ and each edge corresponds to one of the above operations so that at the root reference grid, $\mathcal{R}$, any path to a node corresponds to T. Though the use of a trie here might seem unnatural, it is a useful summary: the trie models all the ways we might apply the spatial operations and a path in the trie models a particular set of operations applied to some starting point. In other words, the trie is the semantic range of our system while a path is what is used at compile time to determine index calculations (or runtime if the operations used to build a view cannot be determined at compile time up to symbolic constants)[7]. Thus, in particular, the trie is an object that we can define our operations on in lieu of a grammar and an associated denotation semantics. The semantics of our operations are given by paths through the trie, in particular via the affine maps between nodes in the path.

The essential content of Theorem 3.1 is a recursively defined trie where each edge has the necessary metadata to define one of the transformations relative to a parent grid. The metadata, as will be seen, allows us to easily compute T^{-1}, but for now we show an example. Figure 3 shows six different 1D and 2D reference space samples assembled into some paths in a UniTe trie. At the top of the trie is the root reference space, which acts as the absolute representation of location; each point in every descendant reference space has a valid absolute location defined within the root (though the converse is not always true, which is why T and T^{-1} must be partial maps). The whole trie is defined recursively, with each reference space defined relative to its parent reference space (except for the root), which creates a sequence of parent-child spatial relationships throughout.

Definition 3.1 (Reference spaces). Let $N \geq 1$ be a natural number. A *reference space*, $\mathcal{R}_c^N$ is an N-dimensional space defined by the 5-tuple $(\mathcal{R}_p^M, \vec{p}, \vec{o}, \vec{r}, \vec{c})$, where

$\mathcal{R}_p^M$ is the parent reference space of $\mathcal{R}_c^N$ $(N \leq M)$,

$\vec{p} = (p_0, \ldots, p_{N-1})$ represents the permutation of $\mathcal{R}_c^N$ with respect to $\mathcal{R}_p^M$ ($0 \leq p_i < M - 1$ and each p_i has a unique value),

[6]This is roughly our colocation operation in Section 4.2.4.

[7]We return to this issue in Section 5 via view access propagation.

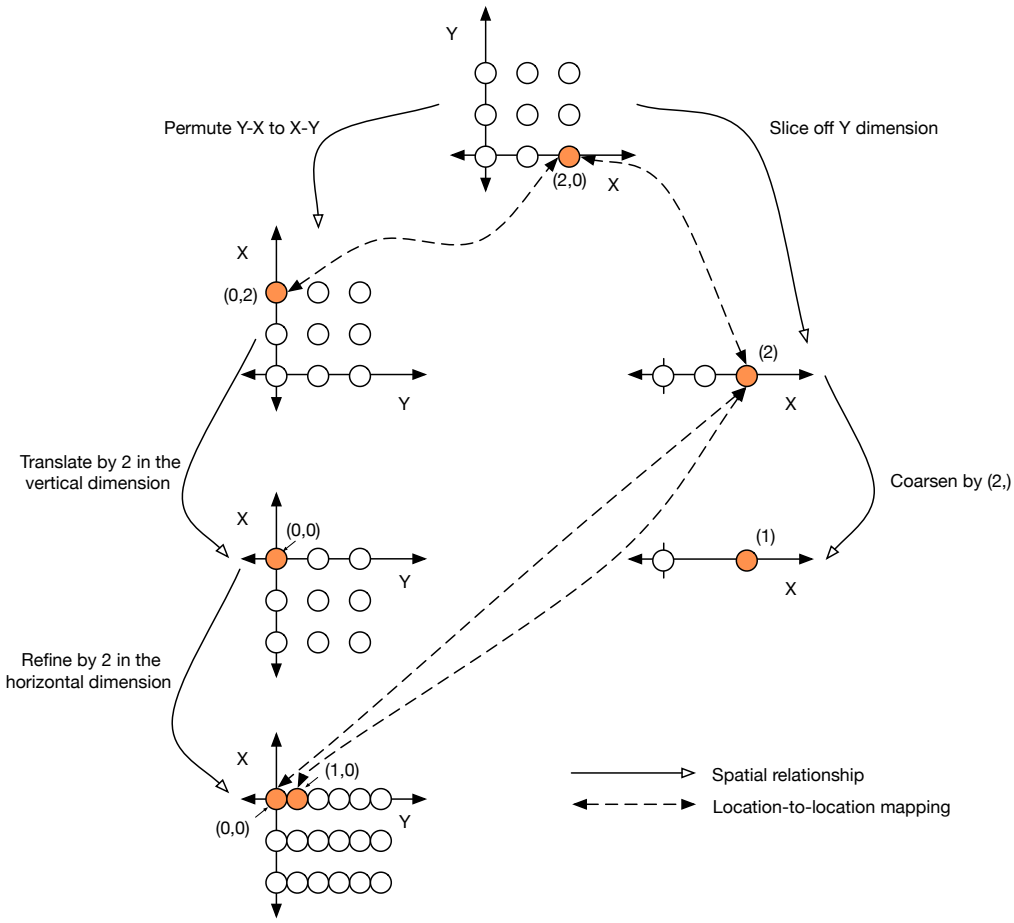


Fig. 3. Some paths in a UniTe trie showing various spatial relationships between parent and child reference spaces, as well as mappings between them. UniTe currently supports five types of relationships (permutation, translation, slicing, refinement, and coarsening), all of which are shown. All the solid colored points represent the same absolute location, despite their varying representations across reference spaces.

$\vec{o} = (o_0, \ldots, o_{M-1})$ represents the origin of $\mathcal{R}_c^N$ with respect to $\mathcal{R}_p^M$ ($o_i \in \mathbb{Z}$),
$\vec{r} = (r_0, \ldots, r_{N-1})$ represents the refinement factor of $\mathcal{R}_c^N$ with respect to $\mathcal{R}_p^M$ ($r_i \in \mathbb{N}$), and
$\vec{c} = (c_0, \ldots, c_{N-1})$ represents the coarsening factor of $\mathcal{R}_c^N$ with respect to $\mathcal{R}_p^M$ ($c_i \in \mathbb{N}$).

In addition, a dimension i cannot be both refined and coarsened (where $r_i > 1$ indicates a refinement and $c_i > 1$ indicates a coarsening), thus r_i and c_i cannot both be greater than one simultaneously. With this definition, *reference space* implies a trie where edges are formed via a reference space's mapping to its parent, and where the root is simply a grid (essentially $\mathbb{Z}^N$). The mappings take a simple form based on the four vectors needed to specify a node in the trie.

Note that UniTe is by no means limited to this set of parameters. For example, some domains can benefit from transformations between coordinate systems, such as going between spherical and rectangular coordinates. However, we find this definition to be broad enough to represent the spatial relationships and primitive operations necessary for

the aforementioned set of application domains. Users of UniTe can add new parameters and primitive operations to UniTe by expanding the definitions as they see fit as long as they ensure we can still correctly and efficiently compute[8] T and T^{-1}. We also note that even if T or T^{-1} are complex, the trie structure admits optimizations still: co-location, $T_1 \circ T_0^{-1}$ can be simplified by using a common ancestor reference space rather than the root of the trie.

3.1.2 Location-to-Location Mappings: the return of T and T^{-1}. A key part of UniTe is that reference spaces, regardless of how they differ from one another, preserve well-defined spatial relationships between the locations in any pair of reference spaces throughout the trie. The UniTe abstraction ensures this by choosing the root of the trie as the absolute spatial location (the location set or root reference space $\mathcal{R}$) so that we simply need to map relative locations in different children to the root via respective T^{-1} mappings to see if they represent the same absolute location. For example, in Figure 3, the solid colored point(s) in each reference all represent the same absolute location seen at the root.

To get to the root from a leaf (to compute T^{-1}), we need show how to move up from a child to the parent space via an upstream mapping that takes sets of points in a reference space and produces a set in the parent. We do this via the upstream mapping defined (Theorem 3.3), which requires an auxiliary definition (Theorem 3.2). By iteratively applying the upstream mapping, we can compute T^{-1}.

Definition 3.2 (Partial permutation matrix). Let P be an $N \times M$ partial permutation matrix defined for a child reference space, $\mathcal{R}_c^N$, with definition $\mathcal{R}_c^N = (\mathcal{R}_p^M, \vec{p}, \vec{o}, \vec{r}, \vec{c})$. Derive P from the permutation vector $\vec{p}$ as follows:

$$P = \begin{bmatrix} P_{0,0} & \cdots & P_{0,M-1} \\ \vdots & \ddots & \vdots \\ P_{N-1,0} & \cdots & P_{N-1,M-1} \end{bmatrix} \qquad P_{i,j} = \begin{cases} 1 & \text{If } p_i = j \\ 0 & \text{Otherwise.} \end{cases}$$

Definition 3.3 (Upstream mapping). Let $\mathcal{R}_c^N$ be a reference space with definition $\mathcal{R}_c^N = (\mathcal{R}_p^M, \vec{p}, \vec{o}, \vec{r}, \vec{c})$. Let $S_{\vec{x}}$ be a set of points where for each $\vec{x} \in S_{\vec{x}}$, $\vec{x}$ is a point within $\mathcal{R}_c^N$. Let P be a partial permutation matrix defined on $\mathcal{R}_c^N$ and $\mathcal{R}_p^M$ as proscribed by $\vec{p}$. An upstream mapping $\phi^{\uparrow}(S_{\vec{x}})$ maps the set $S_{\vec{x}}$ from $\mathcal{R}_c^N$ to $\mathcal{R}_p^M$. Compute the upstream mapping as follows:

$$\phi^{\uparrow}(S_{\vec{x}}) = \bigcup_{\vec{x} \in S_{\vec{x}}} \left\{ P^T(\vec{d} + \vec{i}) + \vec{o} \mid \text{where for each } j \in [0, N),\ d_j = \left\lfloor \frac{c_j x_j}{r_j} \right\rfloor \text{ and } i_j \in [0, c_j) \right\}.$$

We can break this equation down to understand it. First, we are mapping sets so we need to check individual elements and union the results[9]. Second, to handle coarsening and refinement, we compute what box of points a point will land in with $\vec{d}$ and then we iterate though the points in the box with $\vec{i}$. Finally, to handle permutations and slices, we use $P^T(\vec{d} + \vec{i}) + \vec{o}$.

We note that this operation is fairly cheap: $O(N^2 \max_j c_j |S_{\vec{x}}|)$ where N is typically small as high dimensional tensors are rare so the dominant costs are the size of the query and the extent of coarsening; this is as expected as undoing coarsening creates more points to hit but undoing refinement maps many points to one. If we compose this multiple times to form some inverse operation, the cost will be dominated by the largest intermediate cost.

We also need a downstream mapping to map from the parent to the child spaces (to compute T). We can compute the downstream mapping in a manner similar to the above. However, we define the mapping so that we can easily

[8]Users must also consider the cost of implementation and potential optimizations, as additional parameters increase the complexity of index computations, which can decrease performance. Section 5 discusses optimizations based on our definitions of UniTe.
[9]The locations that two points map to should be the union of the locations that each point map to.

claim that the mappings together preserve location[10]. Given points in a parent, we want the downstream mapping to invert the upstream mapping so that locations are preserved. However, since moving from a child to a parent can involve a change in dimension, we can only map and preserve locations that are in the image of the upstream mapping. Thus, the downstream mapping (Theorem 3.4) should behave like an inverse on the image of the upstream mapping, leading to a definition via equations similar to those in the Moore-Penrose pseudo-inverse [Penrose 1955][11].

Definition 3.4 (Downstream mapping). Let $\mathcal{R}_c^N$ be a reference space with definition $\mathcal{R}_c^N = (\mathcal{R}_p^M, \vec{p}, \vec{o}, \vec{r}, \vec{c})$. Let $S_{\vec{x}}$ be a set of points where for each $\vec{x} \in S_{\vec{x}}$, $\vec{x}$ is a point in $\mathcal{R}_p^M$. Similarly, let $S_{\vec{y}}$ be a set of points where for each $\vec{y} \in S_{\vec{y}}$, $\vec{y}$ is a point in $\mathcal{R}_c^N$. A downstream mapping $\phi^{\downarrow}(S_{\vec{x}})$ maps the set $S_{\vec{x}}$ from $\mathcal{R}_p^M$ to $\mathcal{R}_c^N$ by satisfying the equations:

$$\phi^{\downarrow}(\phi^{\uparrow}(\phi^{\downarrow}(S_{\vec{x}}))) = \phi^{\downarrow}(S_{\vec{x}}) \text{ and } \phi^{\uparrow}(\phi^{\downarrow}(\phi^{\uparrow}(S_{\vec{y}}))) = \phi^{\uparrow}(S_{\vec{y}}).$$

These equations directly state that these mappings pseudo-invert each other: down-streaming (up-streaming) is preserved if you upstream (downstream) and then downstream (upstream).

Like the upstream mapping, the downstream mapping (as can be read from the definition in the supplemental), will have runtime $O(N^2 \max_j r_j |S_{\vec{x}}|)$ where N is a small constant. As before, we can compute T via iterative applications of the downstream mapping. And we observe that the equations in the definition of the downstream mapping make it clear that locations in the root referenced with a child space are preserved in other reference spaces, when such locations exist. We also observe the cost of this iterative application will be dominated by the largest intermediate.

4 Building on UniTe

In this section, we build on the UniTe primitives of the last section, introducing several novel operations and associated syntax for creating and accessing tensors based on location. These operations capture much of the functionality necessary to support a practical implementation of UniTe, and in the next sections, we detail different approaches to efficiently implementing the operations introduced here. Table 1 shows all the operations discussed in this section, and includes the associated syntax for our two DSL implementations discussed in later chapters[12]. We indicate in Table 2 how various elements of our formalism are used in the operations; in an interpreter for our system, many operations would just directly use the formalism, but in our suitably restricted language we can determine statically what path in the trie is used and then generate code for that situation.

4.1 Blocks and Views

All the operations described in this section are defined upon two special types of tensors called *block*s and *view*s. Both of these tensors follow the UniTe semantics defined in the prior section, but also contain additional access semantics that better capture data. The primary difference between blocks and views lies in those access semantics, and is intuitively described in the context of an implementation: every block allocates a new region of memory, while a view references an existing region within a parent block. An access to a block returns data within the block's region, while an access to a view propagates through to the block it references.

[10]See supplemental material for an expanded version of the downstream mapping definition.
[11]The same inverse could allow a more computationally direct inversion of the mapping.
[12]We find these operations to provide the capabilities needed for our implementations. However, this is not necessarily a complete set, and users of UniTe in other domains may want to add additional operations as necessary. For example, one potentially useful operation could be creating a new view that points to the overlap between two existing tensors.

Operation	CoLa	SHiM	Description
Create	`b = Block.make(...,type)`	`b = Block<ndims,type>(...)`	Create root block (see (a))
Clone	`b = t.block()`	`b = t.block()`	Clone block from existing tensor
Refine	N/A	$v = t.refine(i_0,...,i_{N-1})$	Apply refinement factors to `t`
Coarsen	See (b)	$v = t.coarsen(i_0,...,i_{N-1})$	Apply coarsening factors to `t`
Slice	See (c)	See (c)	Drop dimensions of `t`
Permute	N/A	$v = t.permute(i_0,...,i_{N-1})$	Reorder dimensions of `t`
Read	$elem = t(i_0,...,i_{N-1})$	$elem = t(i_0,...,i_{N-1})$	Read element at location relative to `t`
Write	$t[i_0,...,i_{N-1}] = elem$	$t[i_0],...,[i_{N-1}] = elem$	Write element at location relative to `t`
Extract	$v = t[x_0:y_0:z_0,...,x_{N-1}:y_{N-1}:z_{N-1}]$	$v = t.extract(\{x_0,y_0,z_0\},...,\{x_{N-1},y_{N-1},z_{N-1}\})$	Extract subtensor from `t`
Colocate	$v = t_0[t_1]$	$v = t_0[t_1]$	Extract subtensor from t_0

v: View, b: Block, t: Tensor (block or view)
(a) "..." represents the initial set of tensor parameters (dimensions, origin, and refinement/coarsening factors)
(b) Coarsening applied within extract
(c) Slice implicitly handled by padding operations

Table 1. Syntax of operations defined on top of UniTe for the two implementations CoLa (Section 6) and SHiM (Section 7).

Operation	Formalism Usage
Create	Creates a reference space root/a one node path in the trie for that root.
Clone	Copies the current path through the trie.
Refine	Creates a new path with a refinement node.
Coarsen	Creates a new path with a coarsen node.
Slice	Creates a new path through a new N and associated origin/partial permutation.
Permute	Creates a new path with a permute node.
Read	Uses the upstream mapping and our current path.
Write	Uses the upstream mapping and our current path.
Extract	Uses the upstream mapping and our current path.
Colocate	Uses the upstream mapping and then the downstream mapping or vice-versa.

Table 2. How operations in our language utilize objects in the formalism.

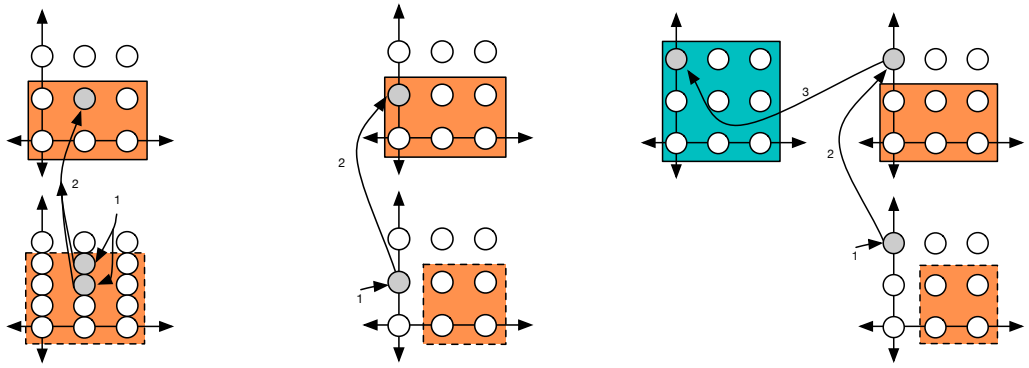
In the context of UniTe, we can discriminate between blocks and views based on their semantics when accessing a locations. Namely, an access to a view always propagates upwards towards a block, while an access on a block either accesses that block directly, or can (optionally) propagate upwards towards another block (see Section 4.2.2). Thus, blocks and views effectively provide bounds on sets of points in a reference space and constrain how far upstream we need to propagate a location. Figure 4a shows an example of an access on a view. Here, the top reference space contains a block (solid line) and the bottom reference space contains a view (dashed line). Accessing the view (arrow labelled 1) propagates to the corresponding location in the block (arrow labelled 2). Since the view's reference space is refined by two in the vertical dimension, both locations in the view map to the same location in the block. If more views exist in the path from the accessed view to the block, the location would propagate through each view until reaching the block.

While blocks and views with simple data access are not novel ideas (e.g. ndarrays and ndarray views in NumPy [Harris et al. 2020]), our use of location allows us to define several novel operations on tops of these tensors that would not be possible in existing implementations. We look at these operations in more detail next.

4.2 Novel Spatial Operations

Here, we define several operations focused on data access and creating new tensors from existing tensors.

4.2.1 Refine, Coarsen, Slice, and Permute. These operations provide an explicit way to control the reference space parameterizations (Theorem 3.1) in which blocks and views exist. We define each such that they return views, regardless of whether the tensor operated on is a block or view. Using the clone operation, a user can create a block with the same parameterization as the view. This ability to create blocks arbitrarily, yet still maintain their spatial relationships, is an important difference between our representation of blocks and views and many other existing implementations, as the others only maintain the origin of a view relative to a single block (e.g. a view in NumPy only maintains its offset into the ndarray it references).

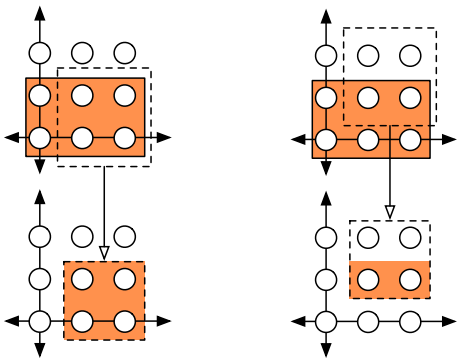


(a) In-bounds (b) Out-of-bounds on view (c) Out-of-bounds on block

Fig. 4. Various access on blocks and views. In (a) and (b), the top reference space contains a block (solid outline), and the bottom one contains a view (dashed outline) with a different parameterization. (c) contains an additional block in the upper reference space (using a different color to indicate it would hold different data). In all cases, accesses propagate up from the view to the block, though (c) contains an additional propagation from block-to-block.

4.2.2 Read and Write Access. The read and write operations access a location in a block, eventually realizing the data at that location and either reading from or writing to it. A read or write access in the context of UniTe is simply an upstream mapping. However, this also provides unique ways of accessing data. First, as discussed in Section 3.1, since the access is an upstream mapping (Theorem 3.3), it supports accessing a view that may have a different reference space parameterization than the associated block. Figure 4a shows one example of this. Here, the view has a refinement, thus multiple locations in the view (arrows labelled 1) map to a single location in the block (arrows labelled 2). This is made possible by maintaining the spatial relationships.

Figure 4b shows a more complex example using an "out-of-bounds" access. Here, the view is accessed beyond its boundary. Even though this access is outside the view, it still represents an in-bounds location relative to the block, so with respect to UniTe, this is just a regular upstream mapping. This type of access provides an intuitive way to access data that surrounds a view (e.g. accessing pixels in the row above and column to the left of a view as in the case study of Section 2).



(a) In-bounds (b) Out-of-bounds

Fig. 5. An in-bounds and out-of-bounds extraction. The dashed boxes in the upper reference space emphasize the region covered by the extraction.

However, these particular semantics are very different from existing implementations. For example, the access on the view (call it v) in Figure 4b can be written as `v(-1,0)` (i.e. `X=-1, Y=0` relative to v); the key here is the negative index. Looking at NumPy again, a negative index on a view counts backwards from the end of the view; it is not possible to do any sort of out-of-bounds access on a view since there is no concept of location in NumPy. It is also possible to perform this type of out-of-bounds

access on a block, which is shown in Figure 4c. Since blocks maintain their locations, we can continually propagate an out-of-bounds access upstream until it reaches an in-bounds location within a block[13].

Out-of-bounds accesses provide an example of the unique types of spatial accesses supported by UniTe's conceptual decoupling of the data of a tensor from the location of a tensor. In this case, all of the intermediate accesses are computed relative to location; only on the final access do we actually realize the data for reading or writing. As discussed in the following examples, these semantics also extend beyond just pointwise access.

4.2.3 Extraction. Extraction creates a new view that references a region within an existing tensor (effectively extracting a subtensor). In both of our implementations, the extraction operations also optionally perform coarsening (the $\mathtt{x_i}$, $\mathtt{y_i}$, and $\mathtt{z_i}$ parameters represent the relative origin, stopping point, and coarsening factor, respectively). Like data access, extractions can also be out-of-bounds, providing unique opportunities for creating views. Figures 5a and 5b show both an in-bounds and out-of-bounds extraction, respectively.

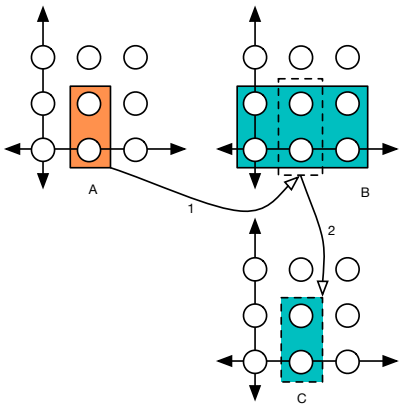


Fig. 6. A colocation. Colocation creates a view (C) on a tensor (B) relative to the location of another tensor (A). Like extractions, colocations can also be performed out-of-bounds.

4.2.4 Colocation. Returning to our discussion in Section 3.1, the colocation operation uses the full power of the abstraction as it requires efficient upstream (T^{-1}) and downstream (T) mappings, and a common root reference space.

It operates similarly to an extraction and creates a view that references a region within an existing tensor. However, colocation extracts the region based on the location of a separate tensor[14]. Figure 6 shows an example of colocation. Here, the location of tensor A is used to access the corresponding region within tensor B, which produces the view C (on B). As with regular extractions, the region produced by colocation can also be out-of-bounds.

5 Compiler Support for UniTe

Using the definitions and descriptions from the last sections, it is straightforward to implement UniTe and its related operations in an unoptimized form. However, due to the extra computation required to access blocks and views (i.e. all reference space parameters must be factored into index computations), such an implementation can incur severe runtime overheads; the overhead can become particularly egregious if accesses happen at the innermost loop levels of a program, causing even small individual overheads to quickly accumulate. The next section discusses the impact of these overheads in more detail and shows that a library implementation runs up to nearly 65× slower than reference C code on a set of compression benchmarks. In this section, we provide a language-agnostic description of two compiler transformations that utilize domain-specific knowledge of UniTe to simplify indexing computations on blocks and views.

[13]In any realistic implementation, an access needs to eventually be in-bounds with respect to a block, otherwise we would access undefined memory.
[14]Thus, colocation is essentially upstream mappings followed by downstream mappings, or roughly $T_1 \circ T_2^{-1}$ in the language of Section 3.1.

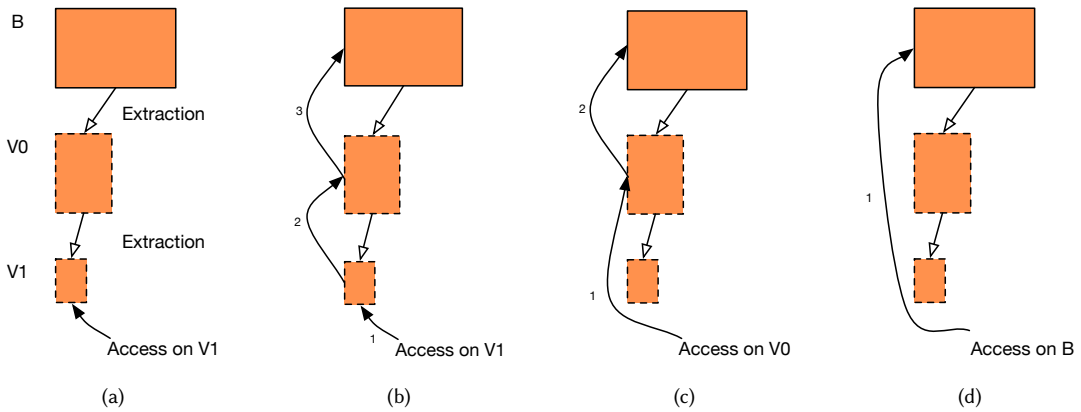


Fig. 7. Depiction of view access propagation. (a) shows the initial structure of blocks and views, and (b) shows the three accesses that would be required at runtime. (c) and (d) show the resulting accesses required after one and two rounds of collapsing, respectively.

The first transformation, *view access propagation*, takes an access on a view and propagates it up to the block ultimately accessed via the view, while *location propagation* performs a domain-specific form of constant and variable propagation that infers values for the spatial parameters of blocks and views. Together, this pair of transformations can simplify the accesses such that they remove extraneous computations (thus matching how the access would be manually written), and remove the need for many of the views. In terms of the formalism, the first transformation determines what paths in the trie have been used while the second simplifies the used paths.

5.1 View Access Propagation

View access propagation moves accesses from views upwards towards blocks. By itself, this transformation does not necessarily optimize performance, as the resulting index computations are the same as what would happen at a runtime. However, it canonicalizes the accesses such that location propagation can better simplify the index computations, and remove the need for many of the views altogether. Another way to think of this is in terms of the formalism: we attempt to determine at compile time what path we have taken through the trie of references space.

Figure 7 gives a simple depiction of view access propagation on a chain of two views and a block. Figure 7a depicts the initial structure of the tensors, while Figure 7b gives the initial configuration, which requires three accesses: one to `V1`, then `V0`, and finally `B`. View access propagation can propagate the access on `V1` to `V0` (Figure 7c), then from `V0` to `B` (Figure 7d).

View access propagation can be broken down into three main steps: 1) identify a view access, 2) find the parent tensor of the view, and 3) propagate the access to the parent. All three steps require domain-specific knowledge in the compiler in order to identify blocks and views, but the final step requires additional knowledge regarding how to recompute an access as it is propagated. In this example, the access on `V1` would be defined relative to `V1`, and propagating it requires recomputing the access relative to `V0`, and then finally relative to `B`.

Figure 8a gives pseudocode showing a write access following the structure of Figure 7, while Figure 8b shows the result of view access propagation. The transformation completely removes the writes on `V1` and `V0`, however, both of those views are still required as their parameters are used in the resulting complex index computation. With the next

transformation, location propagation, it not only simplifies the resulting computation, but also removes the need for `V0` and `V1`.

5.2 Location Propagation

Location propagation performs a domain-specific version of constant and variable propagation on the spatial parameters of blocks and views, helping greatly simplify index computations for accesses. A large source of runtime overhead stems from the index computations as they must take into account all possible parameters, even if the parameters are unused (or are constants). For example, looking back at Figure 8b, the index computations starting on lines 7, 10, 13, and 16 all require a costly division by the refinement factor. However, the refinement factor is a constant in each case: all but `V0.get_refine(1)` are 1, and `V0.get_refine(1)` is 2. Figure 8c shows the resulting index computations after running location propagation.

Looking more into the pseudocode of Figure 8c, there are several components to point out. The most obvious is the use of constant propagation on the spatial parameters, which is the core of the transformation. The ability to perform this propagation requires domain-specific knowledge of blocks and views because the compiler must know how to extract the parameters from each block and view related function. For example, on Line 3, the compiler must have knowledge of how the function `refine(...)` sets the refinement factors of `V0` in order to propagate those values to the calls to `V0.get_refine`. Similarly, on Line 4, the compiler must know how to pull out the origin and coarsening within the extraction for the calls to `V1.get_origin` and `V1.get_coarsen`, respectively. From this, standard constant folding and strength reduction optimizations can further simplify the computations.

Location propagation can also perform variable propagation in the same manner as the constant propagation. In the example, this propagates variables `o0` and `o1` used in the extraction down to the calls to `V1.get_origin`. While this does not enable any further folding or strength reduction, it removes the dependency on `V1`. Combined with the constant propagation, this removes the need for both views, which can be removed with a later dead code elimination pass.

```
B = Block.make(dims=(10,20))                    1
V0 = blk.refine((1,2))                          2
V1 = V0[o0:10:5,o1:20:5]                        3
for i in range(x):                              4
  for j in range(y): V1[i,j] = i + j            5
```

(a) Initial setup

```
x,y,o0,o1 = ...                                 1
B = Block.make(dims=(10,20))                    2
V0 = B.refine((1,2))                            3
V1 = V0[o0:10:5,o1:20:5]                        4
for i in range(x):                              5
  for j in range(y):                            6
    V1_i = V1.get_coarsen(0) * i /              7
           V1.get_refine(0) +                   8
           V1.get_origin(V1.get_perm(0))        9
    V1_j = V1.get_coarsen(1) * j /              10
           V1.get_refine(1) +                   11
           V1.get_origin(V1.get_perm(1))        12
    B_i = V0.get_coarsen(0) * V1_i /            13
          V0.get_refine(0) +                    14
          V0.get_origin(V0.get_perm(0))         15
    B_j = V0.get_coarsen(1) * V1_j /            16
          V0.get_refine(1) +                    17
          V0.get_origin(V0.get_perm(1))         18
    B[B_i,B_j] = i + j                          19
```

(b) After view access propagation

```
x,y,o0,o1 = ...                                 1
B = Block.make(dims=(10,20))                    2
V0 = B.refine((1,2))                            3
V1 = V0[o0:10:5,o1:20:5]                        4
for i in range(x):                              5
  for j in range(y):                            6
    V1_i = 5 * i / 1 + o0                       7
    V1_j = 5 * j / 1 + o1                       8
    B_i = 1 * V1_i / 1 + 0                      9
    B_j = 1 * V1_j / 2 + 0                      10
    B[B_i,B_j] = i + j                          11
```

(c) After location propagation

Fig. 8. Pseudocode for view access propagation and location propagation. View access propagation on (a) recomputes the access on `V1` relative to `B`, producing (b). Location propagation propagates constant and variable location parameters in (c).

6 CoLa: UniTe in Python

This section introduces the standalone DSL and compiler, CoLa (Compression Language), which implements UniTe within the Pythonic language Codon [Shajii et al. 2023].

CoLa provides a general-use implementation of UniTe, and through integration with Codon, users can seamlessly integrate UniTe operations with high-level Pythonic features, providing users with a familiar API. Figure 9 shows two small snippets of CoLa code and highlights the UniTe operations. As expected, CoLa's unoptimized performance incurs severe overheads due to the additional access overhead from UniTe, reaching up to nearly 65× slower than reference C implementations for a set of compression benchmarks. CoLa's optimizing compiler performs view access propagation and location propagation, successfully bringing runtime performance to parity against the reference implementations.

6.1 Integration within Codon

Codon [Shajii et al. 2023] is a high-level, general purpose, statically typed language that implements the syntax of Python, along with many of Python's features and semantics, while also utilizing ahead-of-time compilation.

```
# UniTe: pred/ref are tensors
def intra_4x4_vr(pred, ref):
  # UniTe: colocation
  p = ref[pred]
  for y in range(4):
    for x in range(4):
      zVR = 2*x-y
      if zVR % 2 == 0 and zVR > 0:
        # UniTe: read/writes
        pred[y,x] = (p(-1,x-(y>>1)-1)+ \
                     p(-1,x-(y>>1))+1) \
                     >>1
```

```
# UniTe: tensor is Block or view object
def dct(tensor):
  for r in range(8):
    # UniTe: extract
    row = tensor[r,:]
    # UniTe: read
    tmp0 = int(row(0) + row(7))
    ...
```

Fig. 9. Code snippets showing part of the UniTe API in CoLa.

Codon does not utilize the Python interpreter (or any parts of the standard Python implementation) at any stage of compilation or execution; rather, Codon implements its own full-featured compilation framework that supports parsing, AST/IR generation, typechecking, optimization, and code generation with LLVM. The frontend of CoLa (i.e. user API containing blocks, views, etc.) is written in Codon-proper. To perform the necessary optimizations for CoLa, we modified Codon's compiler and added in CoLa-specific transformation passes[15].

While it is possible to use CoLa purely as a library (i.e. it compiles and runs using the standard Codon compiler), as we will see in Section 6.2, performance is very poor due to the overheads introduced by the accesses on blocks and views.

To combat performance degradation, CoLa implements the view access propagation and location propagation passes and adds them to Codon's compiler. CoLa's implementation for these compiler transformations closely follows their description in the prior section, though there is some slight overlap between the two passes in CoLa, allowing view access propagation to provide some slight performance benefit.

6.2 Evaluation

The optimizations performed in CoLa allow it to both reduce the number of views created (view propagation pass) and specialize location information for tensors (location propagation pass), which impact the performance in different ways. Here, we look at CoLa's performance on three different benchmarks from block-based data compression.

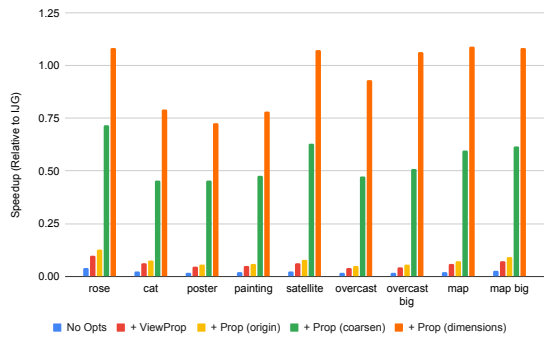
6.2.1 Benchmarks and Experimental Setup. We measure CoLa's performance on two types of JPEG image compression: sequential baseline (JPEGS) and lossless (JPEGL)[16], as well as on the constrained baseline (H.264) profile of H.264 video compression[17]. In each case, we compare performance against the reference implementations included with the

[15]The frontend of CoLa is approximately 700 lines of pure Codon, while CoLa's backend modifications use approximately 7500 lines of C++. CoLa preservers all of the features of the Codon language and compiler.
[16]See Annexes H and F in [CCITT 1992] for descriptions of the sequential baseline and lossless modes, respectively.
[17]See Section A.2.1.1 in [ITU-T 2021a] for a description of the constrained baseline profile.

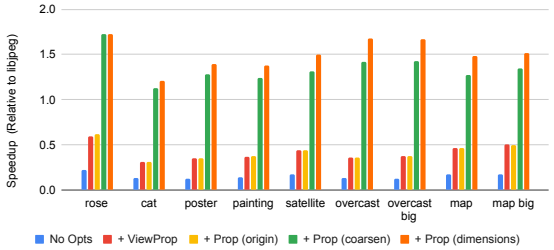
standards, namely IJG [Independent JPEG Group 2022], libjpeg [Richter, Thomas 2022], and JM [Joint Video Team 2022] for JPEGS, JPEGL, and H.264, respectively.

For each benchmark, we first evaluate the performance of CoLa without any of our CoLa-specific optimization passes. This is equivalent to executing the code with Codon-proper, thus represents the performance of Codon alone. This can be considered similar to the performance of a highly-optimized C++ implementation. Then, we add view access propagation ("+ ViewProp"), followed by location propagation for each of the individual parameters ("+ Prop (origin)", "+ Prop (coarsen)", "+ Prop (dimensions)").

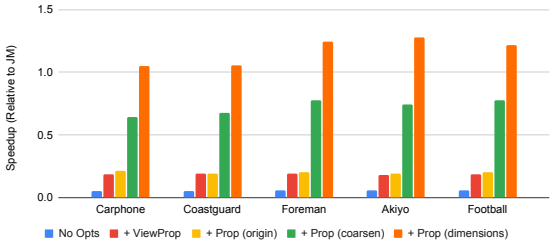
All performance results are computed from the average of 20 iterations. Through Codon, all CoLa code is generated and compiled through LLVM version 12.0.0 with the default -O3 optimizations. All reference codes were compiled with clang 12 (all references are written in C) with -O3, along with the flags included in the Makefiles provided with each reference. Each experiment was run on systems with Intel Xeon E5-2695 v2 cores running at 2.40GHz. All times are for single threaded performance.



(a) JPEGS



(b) JPEGL



(c) H.264

Fig. 10. CoLa's performance relative to the JPEG and H.264 reference implementations. A value of one represents equivalent performance, and values greater than one represent a speedup in CoLa's performance relative to the reference.

6.2.2 Discussion. Figure 10 gives CoLa's performance against the various benchmarks. With the combination of CoLa's optimizations, it achieves average speedups of 1.0×, 1.5×, and 1.2× relative to the reference implementations for JPEGS, JPEGL, and H.264, respectively.

Unsurprisingly, the unoptimized CoLa performance (i.e. baseline Codon) is far worse than compared to the reference C implementations, running nearly 65× slower in the worst case (JPEGS-overcast). There are two main culprits leading to the performance degradation: 1) additional indexing computations and 2) an excess of views. Views are often created at innermost loop levels, and tensors are often indexed at those inner levels as well, thus any additional runtime overhead adds up quickly across all iterations.

As discussed before, location propagation largely targets the overhead caused by the additional indexing computations and exposes many opportunities for constant folding and strength reduction to LLVM. This pass especially helps with extractions, which incur costly overhead due to divisions and multiplications using the parameters. However, in most cases, the divisors are constants, making these

	# Instructions			# Mallocs		
Image	**Pure Library**	**+ ViewProp**	**+ LocProp (all)**	**Pure Library**	**+ ViewProp**	**+ LocProp (all)**
Rose	120,955,600	29,311,077	20,980,925	405,920	1534	642
Cat	1,787,985,652	344,059,715	199,419,046	7,369,600	6,400	642
Poster	16,192,330,494	3,112,528,526	1,665,259,234	74,638,720	17,920	642
Painting	23,307,655,048	5,107,986,425	3,068,869,577	105,269,064	20,482	642
Satellite	66,622,488,297	15,574,441,545	9,765,728,553	300,147,744	29,608	642
Overcast	100,498,604,280	21,404,924,936	12,379,880,192	466,539,040	33,040	642

Table 3. Instruction and `malloc` counts for JPEGL with and without CoLa optimizations.

operations unnecessary. Without location propagation, LLVM misses the opportunities for simplifying these indexing computations.

When creating views, performance is impacted partially due to the fact that Codon allocates objects on the heap, as well as the need for computing the new parameter values of a view whenever one is created. With the two passes, many of these views are eliminated, thus eliminating the associated overheads. Table 3 gives a quantitative example of this elimination and shows the total number of instructions, as well as the total number of mallocs before and after this pass on the JPEGL benchmark. For instruction counts, CoLa is able to reduce the number of instructions by up to a factor of ten, largely due to the fact that removing views also removes the need to recompute a new parameterization for each of them.

Propagation also significantly reduces the number of mallocs in the generated code. Again, this is due to propagation being able to remove many of the views itself. To support Python's pass-by-object-reference semantics, Codon allocates all objects on the heap (and performs automatic memory management). As a result, all blocks and views in CoLa are allocated on the heap as well. This can lead to a bottleneck, as views are often created at the innermost loop levels, requiring a large amount of small allocations, especially when the loop extents are proportional to the input image size (as they are here). As seen in the table, JPEGL can allocate up to nearly 500 million objects. However, after removing a majority of the views created after view propagation, CoLa reduces the number of `mallocs` by over a factor of 14,000. In the end, adding location propagation reduces the number of `mallocs` even further. In the case of JPEGL, all the view allocations happen within the innermost levels of the encoder, and CoLa is able to remove all of them, leaving just the allocations for the initial block objects and some other non-CoLa objects, which all happen outside the main loop nest. As a result, the program ultimately only requires a constant amount of allocations, making allocations invariant to the initial image size.

In the few cases where performance falls below the reference (JPEGS: cat, poster, painting, overcast), we believe the compiler is missing some opportunities for location propagation due to how it extracts the relevant nodes from Codon's IR. We are investigating these opportunities further to see if they exhibit any common patterns that out compiler is missing.

7 SHiM: UniTe in C++

This section describes SHiM (Staged Hierarchical Multidimensional arrays), an embedded C++ DSL and compiler for UniTe[18]. Like CoLa, SHiM targets performance overheads introduced by complex access expressions; however, SHiM does so without the need for explicit compiler passes. Rather, SHiM utilizes staging, which ultimately allows

[18]SHiM is approximately 5700 lines of C++ (not including the BuildIt library).

SHiM to generate fully inlined code that can instead be easily optimized by an existing C++ compiler. Through careful integration with the BuildIt staging library [Brahmakshatriya and Amarasinghe 2021], SHiM allows users to seamlessly interleave UniTe-specific operations with arbitrary control flow without the need for any modifications to an existing C++ compiler.

7.1 Integration within BuildIt

BuildIt [Brahmakshatriya and Amarasinghe 2021] is a C++ library for multi-stage programming that can capture control flow without the need for any changes to C++ or C++ compilers. Unlike Codon, BuildIt is not a standalone DSL or compiler; rather, it is a framework for building DSLs and compilers.

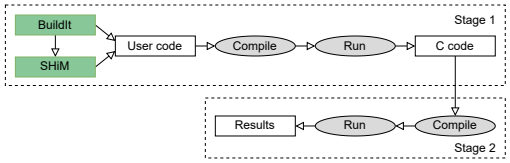


Fig. 11. SHiM's two-stage pipeline layout.

During each stage of execution, BuildIt captures all operations utilizing BuildIt types within an AST and generates semantically equivalent code. SHiM uses standard C++ classes and operator overloading to represent blocks and views, but also stores all parameters for blocks and views in BuildIt types (`dyn_var<T>`, `static_var<T>`, and `dyn_arr<T,N>`), which allows SHiM (through BuildIt) to capture and generate code for all operations performed on blocks and views. SHiM utilizes a two-stage pipeline as shown in Figure 11: users write SHiM code in stage 1, then compile it and run it to generate new C++ code. No UniTe operations are evaluated in stage 1; rather, they are passed through to the generated code in stage 2, then recompiled and evaluated at the end of stage 2. Only stage 2 execution impacts performance.

```
using dint = dyn_var<int>;                                  1
template <typename Pred, typename Ref>                      2
void intra_4x4_vr(Pred &predr, Ref &ref) {                  3
  // UniTe: colocation/permutation                          4
  auto p = ref[predr].permute(1,0);                         5
  // UniTe: permutation                                     6
  auto pred = predr.permute(1,0);                           7
  // UniTe: read/writes                                     8
  for (dint y = 0; y < 4; y=y+1) {                          9
    for (dint x = 0; x < 4; x=x+1) {                        10
      auto zVR = 2*x-y;                                     11
      if (zVR % 2 == 0 && zVR > 0) {                        12
        pred[x][y] = p[x-(y>>1)-1][-1] +                    13
                     p[x-(y>>1)][-1] +                      14
                     1) >> 1;                               15
      ...                                                   16
```

```
template <typename Plane_T>                                 19
void dct1d(Plane_T &plane, dint f) {                        20
  for (dint r = 0; r < 8; r=r+1) {                          21
    // UniTe: extraction                                    22
    auto vec =                                              23
      plane.extract({r,r+1,1},{0,8,1});                     24
    // UniTe: read/writes                                   25
    auto tmp0 = vec(0) + vec(7);                            26
    ...                                                     27
```

Fig. 12. Snippet of the SHiM API.

Combining SHiM with BuildIt gives users the ability to represent control flow and arbitrary arithmetic in their staged program (and other operations supported by BuildIt), while also enabling use of UniTe operations when necessary. Figure 12 shows an example of C++ code written using a combination of SHiM and BuildIt directly. Several UniTe-specific operations in SHiM are shown within for loops using `dyn_var<int>` (`dint`) objects as the loop iterator (Lines 9, 10, and 21); this is one of BuildIt's built-in types (along with `static_var<T>` and `dyn_arr<T,N>`), which users can utilize to capture logic outside of SHiM, such as the for loop.

One of the key differences between the SHiM and CoLa compilers is how SHiM achieves the optimizations described earlier in Section 5. The code generated from stage 1 is fully-inlined; all the underlying array accesses, access computations, and tensor parameters are exposed. This allows SHiM to effectively pass the optimizations to the existing compiler in stage 2, which can perform the necessary constant propagation and folding transformations on the access computations and tensor parameters.

As a result, both SHiM and CoLa ultimately perform the same optimizations; however, SHiM's use of staging through BuildIt allows for a simpler compiler design when compared to CoLa.

The top of Figure 13 shows some simple stage 1 code using extraction and a write within a loop captured by BuildIt. The bottom shows a piece of the generated code. There are two key parts to notice in the generated code:

Tensors are directly exposed The block in stage 1 on Line 3 becomes a simple heap allocated array[19] in the generated code (Lines 24 and 25). The block object itself does not survive through to code generation. Similarly, the view on Lines 4 and 5 in stage 1 does not live through to code generation and instead directly exposes the underlying array for the write on Lines 33 and 34. This effectively performs view access propagation.

Tensor parameters are inlined The generated code highlights some of the inlined tensor parameters, such as the dimensions and coarsening parameters of the block on Lines 15 through 19. Since the parameters are inlined, the existing compiler in stage 2 can easily perform the necessary constant propagation and constant folding optimizations, which helps simplify computations such as computing the block size (Line 23) and dividing by the coarsening factors (Lines 31 and 32).

Thanks to the tight integration between SHiM and BuildIt, these two features of the generated code enable SHiM to leave the "heavy lifting" to the existing compiler without having to sacrifice performance, as we show next.

```
// Stage 1 user code                                          1
void stage1() {                                               2
  auto block = Block<int,2>::heap({10,20});                   3
  auto view = block.extract({0,10,2},                         4
                            {3,19,4});                        5
  for (dint m = 0; m < 5; m=m+1) {                            6
    for (dint n = 0; n < 4; n=n+1) {                          7
      view[m][n] = m + n;                                     8
    }                                                         9
  }                                                           10
}                                                             11
```

```
// Code generated from stage 1                                12
void staged (void) {                                          13
  // Dimensions and coarsen parameters                        14
  int var0 = 10;                                              15
  int var1 = 20;                                              16
  ...                                                         17
  var6 = 1;                                                   18
  var7 = 1;                                                   19
  ...                                                         20
  // Create block                                             21
  int var36 = var0;                                           22
  var36 = var36 * var1;                                       23
  shim::HeapArray<int32_t> var42 =                            24
    shim::build_heaparr<int32_t>(var36);                      25
  ...                                                         26
  // Loop and view write                                      27
  for (int var196=0; var196<5; var196++) {                    28
    for (int var197=0; var197<4; var197++) {                  29
      ...                                                     30
      var312 = (var312 - var8) / var6;                        31
      var313 = (var313 - var9) / var7;                        32
      var42[var313 + (var5 * var312)] =                       33
        var309;                                               34
    }                                                         35
  }                                                           36
}                                                             37
```

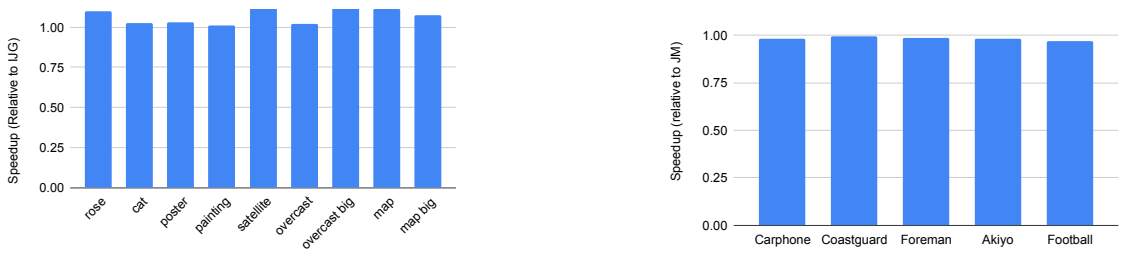
Fig. 13. SHiM stage 1 generates fully inlined code from user code.

7.2 Evaluation

Here we highlight SHiM's performance on the JPEGS and H.264 encoder benchmarks introduced in the prior CoLa section. However, rather than implementing the entire encoder with SHiM as we did in CoLa, we implement various kernels in SHiM and link them into the existing C reference codes, thus replacing the existing functions. In addition, since optimizations are innately integrated into SHiM's overall design (thus cannot be "turned off"), we only generate a single performance point per individual benchmark test (as opposed to incrementally adding on optimizations as with CoLa)[20].

[19] SHiM allows users to specify whether the underlying array generated for a block should be on the heap or the stack, hence the `::heap` function used to create a block.

[20] As stated earlier, BuildIt is a framework for building compilers, so we cannot evaluate the performance of a "BuildIt-only" implementation as we did with Codon and CoLa.



(a) JPEGS (b) H.264

Fig. 14. SHiM's performance relative to the JPEG and H.264 reference implementations.

7.2.1 Discussion. Figures 14a and 14b give the performance of SHiM relative to the IJG and JM reference, respectively. Overall, the results are similar to what was shown for CoLa, with SHiM reaching parity with the reference implementations. The fully inlined nature of the generated code from SHiM allows it to be very easily optimized by an external compiler, thus SHiM, like CoLa, is able to generate code that ultimately removes the overhead associated with the abstraction. More specifically, since blocks and views do not exist in the generated code, all parameter values, as well as access computations, are directly exposed to the stage 2 compiler (clang in this case), which easily performs the necessary optimizations such as constant folding, constant propagation, and strength reduction.

8 Application to Data Movement

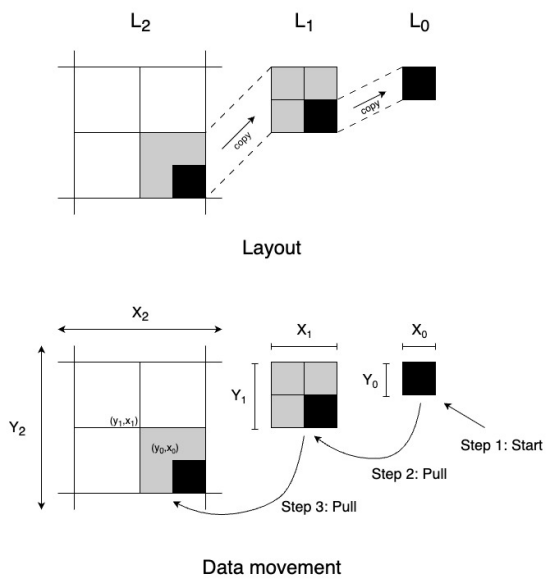
This section highlights how UniTe abstractions can be applied to domains beyond data compression. In particular, we discuss the application of UniTe to data movement within memory hierarchies and distributed layouts.

Consider software-managed memory like those found in GPUs or in contexts such as distributed computing. In either case, data generally does not fit completely into one memory, so it needs to be partitioned and moved around as necessary in order to perform computations. GPUs can have deep memory hierarchies, from general global and shared memories, down to specialized memories like constant and texture. As shown in the layout (top) part of Figure 15a, data must be moved from the higher levels (L_2) of the memory hierarchy (larger memory) to the lowest levels (L_0) of the memory hierarchy (small memory) in order to compute on it. These hierarchies also maintain the containment property that any data contained in level L_q is also contained in level L_{q+1} (ex: data in L_0 is also in L_1 and L_2). With distributed computing, data is partitioned across different memories, or ranks[21]. While there could be overlap in the data within each memory, there is no general containment property like with the memory hierarchy model. Figure 16a gives an example of how the data can be distributed between different memories. Next, we will show how UniTe can apply to data movement operations in both the memory hierarchy and distributed cases.

8.1 Memory Hierarchies

The bottom of Figure 15a annotates tensors in the different memory levels with sizes and origins, showing the tensor in L_2 tiled into tensors that fit into L_1, and the tensors in L_1 tiled into tensors that fit into L_0. Consider the case where we want to compute on all the data in L_2 in a bottom-up manner. In other words, we start with a tile in L_0 and pull in (copy) the data from the corresponding tile in L_1. Similarly, we pull in the corresponding tile from L_2 into L_1. This is a common model of data movement done in an on-demand fashion (i.e. pull data only when it is required, as opposed

[21] Copying from MPI terminology, we differentiate memories with unique rank ids



(a)

```
1  tile_L2 = block(...)
2  def process_tile():
3    // Partition into L0 tiles
4    for y0 = 0 to Y2 by Y0
5      for x0 = 0 to X2 by X0
6        // Create a block for L0's tile
7        tile_L0 = block(tile_L2[y0:y0+Y0,x0:x0+X0])
8        // Get the data from L1.
9        copy_L1_to_L0(tile_L0)
10       // Process it.
11       compute(tile_L0)
12 def copy_L1_to_L0(tile_L0):
13   // Check if the data for L0 already exists in L1.
14   // The exist operation can be done with a simple
15   // check on the bounds
16   if !tile_L1[tile_L0].exists():
17     // Doesn't exist. Pull in a full L1 tile's worth
18     // of data from L2 (in practice, it is usually
19     // more efficient to copy bigger chunks
20     // of data as opposed to many smaller chunks).
21     // Calculate that location, using tile_L0 as the
22     // origin.
23     tile_L1 = block(tile_L0[:Y1,:X1])
24     // get the data from L2
25     copy_L2_to_L1(tile_L1)
26   // Copy a full L0 tile's worth of data
27   // into L0.
28   tile_L0[:,:] = tile_L1[tile_L0][:,:]
29 def copy_L2_to_L1():
30   // Assume that L2 contains all of the data, so we
31   // can just pull a full L1 tile's worth of data
32   // into L1.
33   tile_L1[:,:] = tile_L2[tile_L1][:,:]
```

(b)

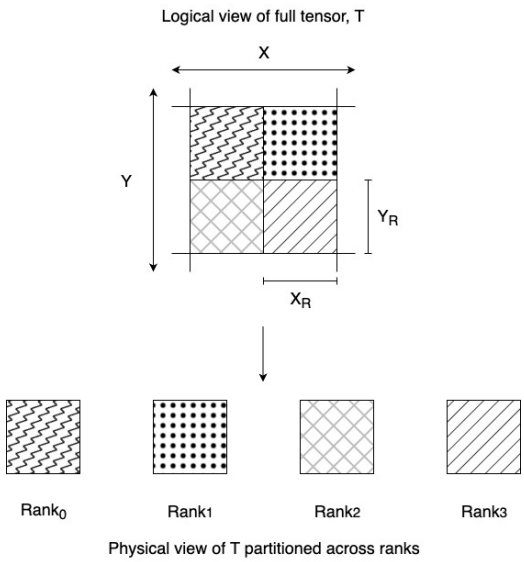
Fig. 15. (a) Data layout (top) across three levels (L_2, L_1, L_0) of a memory hierarchy. Data must be moved to L_0 for computation. In a pull model (bottom), L_0 pulls a tile from L_1, which pulls a tile from L_2. (b) Pseudocode showing how to use UniTe operations to perform the data movement operations highlighted in (a).

to pushing the data before it is needed). Figure 15b shows what this process looks like using UniTe (expressed with pseudocode). The comments in the code detail the high-level semantics, but the key operation to notice is colocation (Lines 28 and 33). Colocation is arguably the most important operation in UniTe because it allows us to map between different tensors (tiles) without having to manually compute their locations. Excluding the creation of the initial blocks, we can perform the fundamental data movement operations for this example with only five UniTe operations.

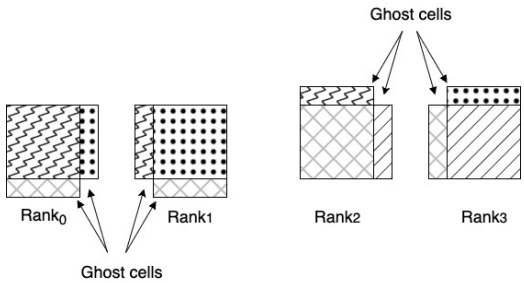
8.2 Distributed

In this example, consider a case where the data has been tiled and already distributed across different memories as shown in Figure 16a. Here, no single rank stores the complete set of data, nor do the ranks exhibit containment as with the hierarchical example. We want to compute on each rank's tile, but we assume the computation requires ghost cells [Kjolstad and Snir 2010], which means each tile requires data stored in a different rank. We use a simple example where each rank needs the data (if available) to the left, right, top, and/or bottom of the tile, relative to the whole logical tensor. Figure 16b shows the ghost cells required in each rank. To perform the computation, each rank must figure out which other rank contains the data corresponding to the ghost cells, and then copy over the data. Again, UniTe provides an intuitive way to do this. Figure 16c shows the pseudocode for the necessary UniTe operations.

As in the memory hierarchy example, colocation is the main operation that lets us operate across the tiles stored in the different ranks (Lines 18, 20, 22, and 24). Note that it does not matter how the tiles are actually partitioned—they



(a) Logical and physical data layout of a tensor, T, distributed across four memory ranks. Here, each rank holds a non-overlapping tile of T.



(b) Ghost cells required for compute (and data transfer).

```
// Assume tiles per rank created apriori
max_rank = ...
tiles_per_rank = map: {rank_id -> tile}
def process_rank(my_rank_id):
  my_tile = tiles_per_rank[my_rank_id]
  // Create empty blocks to store ghost cells
  top = block(my_tile[-1,:])
  left = block(my_tile[:,-1])
  down = block(my_tile[YR,:])
  right = block(my_tile[:,XR])
  // Go through each of the ranks and figure
  // out where the ghost cell data resides
  for r = 0 to max_rank:
    if r != my_rank_id:
      other_tile = tiles_per_rank[r]
      // If find the data, copy it
      if other_tile[top].exists():
        top[:,:] = other_tile[top][:,:]
      if other_tile[left].exists():
        left[:,:] = other_tile[left][:,:]
      if other_tile[down].exists():
        down[:,:] = other_tile[down][:,:]
      if other_tile[right].exists():
        right[:,:] = other_tile[right][:,:]
  // Process the tile with the ghost cells
  compute(my_tile, top, left, down, right)
```

(c) Pseudocode showing how to use UniTe operations to perform the data movement operations for ghost cells highlighted in (a) and (b).

Fig. 16. Example breakdown of distributed data movement with UniTe.

could be overlapping, uneven sizes, strided, and so on, but the UniTe abstraction still just works. Finally, while the details of how the actual data copy happens in these examples are not considered in this paper, an implementation can easily generate the necessary copy operations (e.g. use of `MPI_Send`, `MPI_Recv` for MPI or `__shared__` in CUDA) from a UniTe abstraction layer.

9 Related Works

Focusing on the abstraction, UniTe relates most to prior work in the domain of array programming. Array-based languages and libraries sharing some similarities of UniTe include Julia [Bezanson et al. 2017], APL [Iverson 1962], Ada [Barnes 1984], MATLAB [Higham and Higham 2016], NumPy [Harris et al. 2020], Eigen [Guennebaud et al. 2010], Blaze [Blaze 2023], Blitz++ [blitzpp 2023], SaC [Scholz 2003], J [Iverson 2000], Armadillo [Sanderson and Curtin 2016], and FTensor [Landry 2023], Halide [Ragan-Kelley et al. 2013b], and Triton [Tillet et al. 2019]. While these tools

individually capture some of the concepts in UniTe (such as having views) none provide the full abstraction necessary to capture and reason about location and spatial relationships. As an example, in Halide, even if two functions map to the same underlying buffer, there is no colocation available that would allow users to perform an operation such as copying between functions (without the use of manual indexing, which defeats the purpose). A similar analysis holds for Triton tensors. Similarly, tools like NumPy and Julia provide native support for views and various types of manipulation (slicing, extraction, etc.), but do not expose any of the location information for views related to their blocks, nor do they provide a global concept of location. This limits their ability to perform colocation. Since these tools do not track relationships between multiple arrays (as opposed to just one array is a view/reshape of another), users must manually track spatial relationships and write corresponding index expressions. Worse, array libraries exemplified by NumPy provide users with indexing building blocks[22] that users could use to build an indexing formula for a particular colocation, but besides being error-prone, these often realize unnecessary intermediate tensors to do the index calculation. Compiler based tools that help with numpy-esq arrays (such as Codon[Shajii et al. 2023] or Julia[Bezanson et al. 2017]) do reduce this overhead, but the manual effort remains.

Other systems with multidimensional arrays focus on the view/hierarchical aspect, which is more closely aligned with this work. Marray [Andres et al. 2010] provides a hybrid templated/runtime library implementation in C++ that treats strided views as a core primitive. Velociraptor [Garg and Hendren 2014] provides a framework for building JIT compilers for multidimensional tensors and views, providing an array-based IR. The work on Flexible Data Views [Osvald and Rompf 2017] utilizes templates for C++ and dynamic staging through Lightweight Modular Staging [Rompf and Odersky 2010] for Scala and focuses on providing views that cover disjoint regions of data. Hierarchically tiled arrays [Bikshandi et al. 2006, 2007; Fraguela et al. 2004] provide a way to partition strided data into tiles and distribute the tiles across memories. Taichi [Hu et al. 2019] focuses on the hierarchical structure of disjoint and sparse data. Finally, the View Template Library [Powell and Weiser 1998] represents views in a functional way with smart iterators [Becker 1998]. However, none of these previous works capture spatial relationships in a way that can be leveraged to support the more general reference space mappings provided by UniTe.

In more loosely related works, some DSLs have adopted a notion of reference spaces to address similar problems on non-array abstractions. For example, Gator [Geisler et al. 2020] offers a no-overhead coordinate system type to ensure vectors in graphics programs belong to the correct coordinate system. Also CoolerSpace [Chen et al. 2024] offers a similar abstraction for different representations of color (similar to coordinate systems) though here the focus is on optimizing ways to go between different colors through various intermediate options. Similarly, Diderot [Chiw et al. 2012; Collin et al. 2019] offers tensor fields that have a coordinate system derived from raster data via the NRRD format [Kindlmann [n. d.]]. Finally, several DSLs for numerical methods, especially Galerkin methods, use notions of reference spaces to deduce parts of the simulation [Bastian et al. 2021; Logg et al. 2012].

10 Conclusion

Computations in many domains are fundamentally built off of tensors that maintain spatial relationships amongst one another, and complexity arises in implementations due to the required transformations between the mathematical spaces in which the tensors reside. Variations in the representations of these spatially related tensors, such as their relative dimensionality, orderings, origins, and densities all must be accounted for in these transformations, leading to complex index calculations and bookkeeping that users must manually perform. Not only does these lead to overly

[22] see https://numpy.org/doc/stable/reference/routines.indexing.html

complicated and bug-prone code, but users must additionally ensure that these computations are performant, as they often occur at the innermost loop level of programs, meaning even small overheads add up quickly. As a result, next to no programming language (or library) support exists for these types of spatially related tensors.

This paper presents the first work targeting these types of tensors. We first introduced UniTe, a mathematical abstraction that captures the spatial relationships across tensors, and then defined several unique operations built from UniTe primitives. Then, we presented two domain-specific languages and compilers, CoLa and SHiM, which implement the abstraction and optimize the underlying access computations. Together, UniTe, CoLa, and SHiM show that it is possible to provide users an intuitive framework for maintaining and using these types of tensors, without sacrificing performance compared to manual C implementations. We believe the programming language and compiler extensions we described in this work are general and intuitive, lending themselves well to further use in other modern languages, thus expanding UniTe's reach.

References

Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2015. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. https://www.tensorflow.org/ Software available from tensorflow.org.

Bjoern Andres, Ullrich Koethe, Thorben Kroeger, and Fred A Hamprecht. 2010. Runtime-flexible multi-dimensional arrays and views for C++ 98 and C++ 0x. *arXiv preprint arXiv:1008.2909* (2010).

John Backus. 1978. The history of Fortran I, II, and III. *ACM Sigplan Notices* 13, 8 (1978), 165–180.

John G Barnes. 1984. *Programming in ADA*. Addison-Wesley Longman Publishing Co., Inc.

Peter Bastian, Markus Blatt, Andreas Dedner, Nils-Arne Dreier, Christian Engwer, René Fritze, Carsten Gräser, Christoph Grüninger, Dominic Kempf, Robert Klöfkorn, et al. 2021. The Dune framework: Basic concepts and recent developments. *Computers & Mathematics with Applications* 81 (2021), 75–112.

Thomas Becker. 1998. Smart iterators and stl. *C/C++ Users Journal* 16, 9 (1998), 39–45.

Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev.* 59, 1 (Jan. 2017), 65–98. doi:10.1137/141000671

Ganesh Bikshandi, Jia Guo, Daniel Hoeflinger, Gheorghe Almasi, Basilio B Fraguela, María J Garzarán, David Padua, and Christoph Von Praun. 2006. Programming for parallelism and locality with hierarchically tiled arrays. In *Proceedings of the eleventh ACM SIGPLAN symposium on Principles and practice of parallel programming*. 48–57.

Ganesh Bikshandi, Jia Guo, Christoph Von Praun, Gabriel Tanase, Basilio B Fraguela, María J Garzarán, David Padua, and Lawrence Rauchwerger. 2007. Design and use of htalib–a library for hierarchically tiled arrays. In *Languages and Compilers for Parallel Computing: 19th International Workshop, LCPC 2006, New Orleans, LA, USA, November 2-4, 2006. Revised Papers 19*. Springer, 17–32.

Blaze. 2023. blaze-lib. "https://bitbucket.org/blaze-lib/blaze/src/master/"

blitzpp. 2023. blitz. "https://github.com/blitzpp/blitz"

Ajay Brahmakshatriya and Saman Amarasinghe. 2021. BuildIt: A type-based multi-stage programming framework for code generation in C++. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 39–51.

CCITT. 1992. *Information Technology–Digital Compression and Coding of Continuous-Tone Still Images–Requirements and Guidelines*. Technical Report T.81. International Telecommunication Union, Geneva, Switzerland.

Ethan Chen, Jiwon Chang, and Yuhao Zhu. 2024. Coolerspace: A language for physically correct and computationally efficient color programming. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024), 846–875.

Charisee Chiw, Gordon Kindlmann, John Reppy, Lamont Samuels, and Nick Seltzer. 2012. Diderot. *ACM SIGPLAN Notices* 47, 6 (June 2012), 111–120. doi:10.1145/2345156.2254079

Teodoro Collin, Charisee Chiw, L. Ridgway Scott, John Reppy, and Gordon Kindlmann. 2019. Point Movement in a DSL for Higher-Order FEM Visualization. In *2019 IEEE Visualization Conference (VIS)*. 281–285. doi:10.1109/VISUAL.2019.8933623

Basilio B Fraguela, Jia Guo, Ganesh Bikshandi, Maria J Garzaran, Gheorghe Almasi, Jose Moreira, and David Padua. 2004. The hierarchically tiled arrays programming approach. In *Proceedings of the 7th workshop on Workshop on languages, compilers, and run-time support for scalable systems*. 1–12.

Rahul Garg and Laurie Hendren. 2014. Velociraptor: An embedded compiler toolkit for numerical programs targeting cpus and gpus. In *Proceedings of the 23rd international conference on Parallel architectures and compilation*. 317–330.

Dietrich Geisler, Irene Yoon, Aditi Kabra, Horace He, Yinnon Sanders, and Adrian Sampson. 2020. Geometry types for graphics programming. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (Nov. 2020), 1–25. doi:10.1145/3428241

Gaël Guennebaud, Benoit Jacob, et al. 2010. Eigen. *URl: http://eigen. tuxfamily. org* 3 (2010).

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. 2020. Array programming with NumPy. *Nature* 585, 7825 (Sept. 2020), 357–362. doi:10.1038/s41586-020-2649-2

Desmond J Higham and Nicholas J Higham. 2016. *MATLAB guide*. SIAM.

Yuanming Hu, Tzu-Mao Li, Luke Anderson, Jonathan Ragan-Kelley, and Frédo Durand. 2019. Taichi: a language for high-performance computation on spatially sparse data structures. *ACM Transactions on Graphics (TOG)* 38, 6 (2019), 1–16.

Independent JPEG Group. 2022. JPEG software. https://ijg.org/

ITU-T. 2021a. *Advanced Video Coding for Generic Audiovisual Services*. Technical Report H.264. International Telecommunication Union, Geneva, Switzerland.

ITU-T. 2021b. *High Efficient Video Coding*. Technical Report H.265. International Telecommunication Union, Geneva, Switzerland.

ITU-T. 2022. *Versatile Video Coding*. Technical Report H.266. International Telecommunication Union, Geneva, Switzerland.

Kenneth E Iverson. 1962. A programming language. In *Proceedings of the May 1-3, 1962, spring joint computer conference*. 345–351.

K. E. Iverson. 1991. A personal view of APL. *IBM Systems Journal* 30, 4 (1991), 582–593. doi:10.1147/sj.304.0582

Kenneth E Iverson. 2000. A personal view of APL. *ACM SIGAPL APL Quote Quad* 30, 3 (2000), 4–13.

Joint Video Team. 2022. JM software (v19.0). http://iphome.hhi.de/suehring/

Gordon Kindlmann. [n. d.]. NRRD: Definition of NRRD file format. https://teem.sourceforge.net/nrrd/format.html

Fredrik Berg Kjolstad and Marc Snir. 2010. Ghost cell pattern. In *Proceedings of the 2010 Workshop on Parallel Programming Patterns*. 1–9.

Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. 2015. Numba: A llvm-based python jit compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*. 1–6.

Walter Landry. 2023. ftensor. "https://wlandry.net/Projects/FTensor/"

Anders Logg, Kent-Andre Mardal, and Garth Wells (Eds.). 2012. *Automated Solution of Differential Equations by the Finite Element Method*. Springer Berlin Heidelberg. doi:10.1007/978-3-642-23099-8

Henry Margenau and George Moseley Murphy. 2009. *The mathematics of physics and chemistry*. Young Press.

D Maydan, S Amarsinghe, and M Lam. 1993. Data dependence and data-flow analysis of arrays. In *Languages and Compilers for Parallel Computing: 5th International Workshop New Haven, Connecticut, USA, August 3–5, 1992 Proceedings 5*. Springer, 434–448.

J. Oberg. 1999. Why the Mars probe went off course [accident investigation]. *IEEE Spectrum* 36, 12 (1999), 34–39. doi:10.1109/6.809121

Leo Osvald and Tiark Rompf. 2017. Flexible data views: design and implementation. In *Proceedings of the 4th ACM SIGPLAN International Workshop on Libraries, Languages, and Compilers for Array Programming*. 25–32.

R. Penrose. 1955. A generalized inverse for matrices. *Mathematical Proceedings of the Cambridge Philosophical Society* 51, 3 (July 1955), 406–413. doi:10.1017/s0305004100030401

Gary Powell and Martin Weiser. 1998. Views, A New Form of Container Adaptors. *C/C++ Users Journal* (04 1998).

William Pugh. 1992. A practical algorithm for exact array dependence analysis. *Commun. ACM* 35, 8 (1992), 102–114.

Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013a. Halide. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM. doi:10.1145/2491956.2462176

Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013b. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices* 48, 6 (2013), 519–530.

Richter, Thomas. 2022. libjpeg. https://github.com/thorfdbg/libjpeg

Tiark Rompf and Martin Odersky. 2010. Lightweight modular staging: a pragmatic approach to runtime code generation and compiled DSLs. In *Proceedings of the ninth international conference on Generative programming and component engineering*. 127–136.

Conrad Sanderson and Ryan Curtin. 2016. Armadillo: a template-based C++ library for linear algebra. *Journal of Open Source Software* 1, 2 (2016), 26.

Sven-Bodo Scholz. 2003. Single Assignment C: efficient support for high-level array operations in a functional setting. *Journal of functional programming* 13, 6 (2003), 1005–1059.

Ariya Shajii, Gabriel Ramirez, Haris Smajlović, Jessica Ray, Bonnie Berger, Saman Amarasinghe, and Ibrahim Numanagić. 2023. Codon: A Compiler for High-Performance Pythonic Applications and DSLs. In *Proceedings of the 32nd ACM SIGPLAN International Conference on Compiler Construction*. 191–202.

Max Taylor, Jayson Boubin, Haicheng Chen, Christopher Stewart, and Feng Qin. 2021. A Study on Software Bugs in Unmanned Aircraft Systems. In *2021 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE. doi:10.1109/icuas51884.2021.9476844

Philippe Tillet, Hsiang-Tsung Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. 10–19.

Sven Verdoolaege. 2010. isl: An integer set library for the polyhedral model. In *International Congress on Mathematical Software*. Springer, 299–302.

Peng Wu. 2023. PyTorch 2.0: The Journey to Bringing Compiler Technologies to the Core of PyTorch (Keynote). In *Proceedings of the 21st ACM/IEEE International Symposium on Code Generation and Optimization*. ACM. doi:10.1145/3579990.3583093